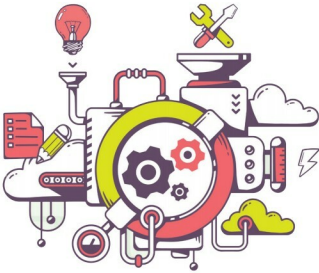


A Glimpse of Microservices with Kubernetes and Docker

The microservices architecture is a variant of service oriented architecture. It develops a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API.



‘Microservices’ is a compound word made of ‘micro’ and ‘services’. As the name suggests, microservices are the small modules that provide some functionality to a system. These modules can be anything that is designed to serve some specific function. These services can be independent or interrelated with each other, based on some contract.

The main function of microservices is to provide isolation between services — a separation of services from servers and the ability to run them independently, with the interaction between them based on a specific requirement. To achieve this isolation, we use containerisation, which will be discussed later. The idea behind choosing microservices is to avoid correlated failure in a system where there is a dependency between services. When running all microservices inside the same process, all services will be killed if the process is restarted. By running each service in its own process, only one service is killed if that process is restarted, but restarting the server will kill all services. By running each service on its own server, it’s easier to maintain these isolated services, though there is a cost associated with this option.

How microservices are defined

The microservices architecture develops a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API.

Microservices is a variant of the service-oriented architecture (SOA) architectural style. A SOA is a style of software design in which services are provided to the other components by application components, through a communication protocol over a network that structures an application as a collection of loosely coupled services. In the microservices architecture, services should be fine-grained and the protocols should be lightweight. The benefit of breaking down an application into different smaller services is that it improves modularity and makes the application easier to understand, develop and test. It also parallelises development by enabling small autonomous teams to develop, deploy and scale their respective services independently.

These services are independently deployable and scalable. Each service also provides a kind of contract allowing for different services to be written in different programming languages. They can also be managed by different teams.

The architecture of microservices

Microservices follows the service-oriented architecture in which the services are independent of users, products and technologies. This architecture allows one to build applications as suites of services that can be used by other services. This architecture is in contrast to the monolithic architecture, where the services are built as a single unit comprising a client-side user interface, databases and server-side applications in a single frame — all dependent on one another. The failure of one can bring down the whole system.

The microservices architecture mainly consists of the client-side user interface, databases and server-side applications as different services that are related in some way to each other but are not dependent on each other. Each layer is independent of the other, which in turn leads to easy maintenance. The architecture is represented in Figure 2.

This architecture is a form or system that is built by plugging together components, somewhat like in a real world composition where a component is a unit of software that is independently replaceable and upgradeable. These microservices are easily deployable and integrated into one another. This gives rise to the possibility of continuous integration and continuous deployment.